BNP/1

Binary Network Protocol, version 1 - static file fetch over TCP

1. Purpose and connection model

BNP/1 carries one static-file GET request and its response in binary frames. Start a server with **bserve ROOT PORT**; fetch with **bcurl [-v] HOST:PORT/PATH**. A client creates exactly one TCP connection for its requested URL and never opens a second connection. A connection may carry further ordered request/reply exchanges until a peer closes it or a framing-level error prevents safe resynchronisation.

All multibyte integers are unsigned big-endian. Text is UTF-8 unless stated otherwise. The implementation accepts payloads through 8,388,608 bytes and carries arbitrary body bytes without text conversion.

2. Frame layer

Offset	Width	Field	Meaning
0	2	magic	ASCII BN (42 4e)
2	1	version	1 for this specification
3	1	type	1 request; 2 response; other values are extensions
4	1	flags	zero in v1; receiver ignores it
5	1	reserved	MUST be zero
6	2	stream ID	client-chosen nonzero correlation value
8	4	payload length	number of following payload bytes

The fixed header establishes exactly where a frame ends. Magic rejects accidental traffic; version and type permit evolution. The 16-bit stream ID supports correlation, and a 32-bit declared length supports binary files while the 8 MiB cap limits allocation. Flags and one reserved byte leave defined space for v2.

Extension rule: a receiver meeting a frame type it does not know MUST read and discard exactly its declared payload length, then continue with the next frame. It must not inspect unknown fields or close a sound connection merely because the type is unfamiliar.

If magic, version, reserved byte, length limit, or a complete payload cannot be read, framing is malformed and the connection closes because boundaries cannot be trusted. A known request frame with a bad payload receives 400 on its stream and the connection stays usable.

3. Header representation

After a request path or response status, headers are **count:u8**, then count entries. Each starts with **name_id:u8**. IDs 1-10 name: 1 host, 2 content-type, 3 content-length, 4 connection, 5 date, 6 server, 7 accept, 8 user-agent, 9 cache-control, 10 last-modified. An indexed entry continues **value_length:u16, value:utf8**.

For another name, send **name_id=0, name_length:u8, name:ASCII, value_length:u16, value:UTF-8**. Literal names are nonempty. Indexing stable common names saves bytes; length-prefixed literals allow extensions without a registry. Unknown nonzero name IDs are malformed in v1 because their names are not self-delimiting.

BNP/1 - Specification (continued)

4. Request and response payloads

A request (type 1) is **method:u8, path_length:u16, path:UTF-8, headers**. Method 1 is GET and is the only valid method. A path starts with /, has no query or fragment, and has no empty, . or .. component. The server resolves it beneath its configured root; a missing regular file is 404.

A response (type 2) is **status:u16, headers, body_length:u32, body**. body_length consumes the remaining payload exactly. The server sends content-type, content-length and connection: keep-alive. It returns 200 for a found file, 404 when absent, 400 for a well-framed bad request, and 500 for an internal file error. bcurl writes body bytes only to stdout; -v writes decoded metadata and frame hexdumps to stderr. It exits 0 for a status below 400, 4 for 4xx, and 5 for 5xx.

5. Receiver requirements

Read TCP until every declared byte arrives; recv calls are not frame boundaries. Before allocation, check every length against the remaining payload and implementation cap. Reject invalid UTF-8, invalid header IDs, empty literal header names, trailing bytes in a known payload, invalid status, and root-escaping paths. These checks prevent desynchronisation and unsafe file access.

Known malformed request payloads produce a type-2 response with status 400. Unknown frame types are skipped only after all their declared bytes have been consumed. This is the compatibility mechanism that lets a later protocol version add frame types without breaking an older peer.

6. Interoperability example

For **localhost:9000/index.html**, bcurl sends a type-1 stream-1 frame whose request payload is GET (01), path /index.html, indexed host localhost:9000, and indexed user-agent bcurl/1. bserve answers on stream 1 with status 200, indexed content-type text/html, indexed content-length, indexed connection keep-alive, then the exact file bytes. The submitted annotated hexdump records every byte of this exchange.

Implementation checklist

Client	Build one binary request frame, read one response, send body to stdout, hexdump frames with -v, use no second connection, return nonzero for 4xx/5xx.
Server	Accept TCP, read complete binary frames despite segmentation, safely map paths under root, return status/headers/body, 404 missing files, 400 malformed known requests, and preserve usable connections.
Protocol	Fixed 12-byte header, numbered first ten header names, length-prefixed remaining names, checked lengths, and mandatory unknown-frame skip rule.